RPM-MCTS: Knowledge-Retrieval as Process Reward Model with Monte Carlo Tree Search for Code Generation

Yuanyuan Lin[1,3], **Xiangyu Ouyang**[2], **Teng Zhang**[1*], **Kaixin Sui**[3]
[1] School of Computer Science and Technology, Huazhong University of Science and Technology, China
[2] College of Computer Science and Technology, Xi'an Jiaotong University, China
[3] ByteDance Seed, China
{linyy, tengzhang}@hust.edu.cn, oyxy2019@stu.xjtu.edu.cn, suikaixin@163.com

Abstract

Tree search-based methods have made significant progress in enhancing the code generation capabilities of large language models. However, due to the difficulty in effectively evaluating intermediate algorithmic steps and the inability to locate and timely correct erroneous steps, these methods often generate incorrect code and incur increased computational costs. To tackle these problems, we propose RPM-MCTS, an effective method that utilizes Knowledge-Retrieval as Process Reward Model based on Monte Carlo Tree Search to evaluate intermediate algorithmic steps. By utilizing knowledge base retrieval, RPM-MCTS avoids the complex training of process reward models. During the expansion phase, similarity filtering is employed to remove redundant nodes, ensuring diversity in reasoning paths. Furthermore, our method utilizes sandbox execution feedback to locate erroneous algorithmic steps during generation, enabling timely and targeted corrections. Extensive experiments on four public code generation benchmarks demonstrate that RPM-MCTS outperforms current state-of-the-art methods while achieving an approximately 15% reduction in token consumption. Furthermore, full fine-tuning of the base model using the data constructed by RPM-MCTS significantly enhances its code capabilities.

Introduction

Code generation aims at understanding problem descriptions in natural language and generating the corresponding code snippets. In recent years, large language models (LLMs) have demonstrated remarkable performance in code generation tasks (Zhang et al. 2024b). For code generation, the early methods involve dividing code planning and synthesis into two phases using chain-of-thought or tree structures (Wei et al. 2022; Jiang et al. 2024; Zelikman et al. 2023). Wang et al. (2024) have demonstrated that providing LLMs with *a correct* solution can significantly enhance model performance, even when these solutions consist of incomplete plans, i.e., for solutions, correctness is preferred over completeness, and the key to enhancing the code generation capability of LLMs lies in generating correct plans.

Programming languages possess their own inherent logical structures and tightly interconnected knowledge, which makes it essential not to overlook long-range dependencies within the code. Previous work has shown that rotary position embedding does not always lead to attention weights decaying with relative distance (Barbero et al. 2024). Concurrently, through exploring the attention distribution between tokens, we have experimentally demonstrated that selecting algorithmic steps as the basic units is a superior choice. Therefore, our objective focuses on how to accurately generate intermediate algorithmic steps.

However, a limitation of previous methods lies in the lack of an evaluation and correction mechanism for intermediate algorithmic steps, which fails to guarantee the correctness of these steps (Lu et al. 2025; Li et al. 2025a). One way to tackle this issue is to use a value function or reward model to verify reasoning traces for correctness, which then serves as a learning signal for self-training (Lightman et al. 2024; Wang et al. 2023). However, training a reliable reward model to verify every step in a reasoning trace generally depends on dense human-generated annotations per reasoning step (Lightman et al. 2024), which does not scale well.

Unlike other reasoning tasks, code generation benefits from the homogeneity of algorithmic workflows across different problem categories. This allows us to leverage historical experience from a knowledge base containing numerous correct algorithmic steps to evaluate the process reward of expansion steps. Additionally, code generation typically benefits from detailed feedback provided by compilers. Consequently, in this paper, we propose RPM-MCTS, which optimizes the Monte Carlo Tree Search (MCTS) algorithm using external information feedback. Our method utilizes the knowledge base for intermediate algorithmic step-level evaluation and employs sandbox feedback for result-level assessment of complete code. Specifically, the root node of the Monte Carlo tree represents the coding problem, while all other nodes represent individual algorithmic steps. During each iteration, multiple distinct potential next steps are generated based on the current reasoning path. Node selection is guided by historical experience from the knowledge base, enabling faster discovery of high-value search paths. In the simulation phase, complete code is generated and evaluated using sandbox and model feedback to update node values. Notably, during simulation, we localize erroneous steps within the full algorithmic workflow and incorporate newly generated correct steps into the tree, thereby reducing token

*Corresponding Author